



UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Memoria de Práctica

Lab-MR 1-I

Seguimiento de caminos explícitos

Navegación punto a punto con control proporcional

Ampliación de Robótica

Saúl Gutiérrez Rodríguez

Pablo Haro García

Mario Merino Prado

Repositorio: github.com/marioomerino24/ampliacion_robotica

Entorno: ROS 2 Humble · Coppeliasim · Pioneer 3-DX

Índice

1. Introducción	3
1.1. Objetivo de la práctica	3
1.2. Descripción del problema	3
2. Fundamentos teóricos	3
2.1. Modelo cinemático del robot	3
2.2. Algoritmo de control: control proporcional de rumbo	4
3. Diseño e implementación	5
3.1. Arquitectura del sistema	5
3.2. Estrategia de control	5
3.3. Parametrización	6
4. Resultados experimentales	6
4.1. Escenarios de prueba	6
4.2. Análisis cuantitativo	6
4.3. Análisis cualitativo	7
5. Discusión	7
5.1. Problemas encontrados y soluciones	7
5.2. Limitaciones	8
6. Conclusiones	8

1. Introducción

1.1. Objetivo de la práctica

El objetivo de esta práctica es implementar un sistema de **navegación punto a punto** (*point-to-point*, P2P) para un robot Pioneer P3DX simulado en CoppeliaSim. El robot debe recorrer de forma autónoma una secuencia de waypoints predefinidos, siguiendo un camino **explícito** —es decir, la trayectoria se define mediante coordenadas conocidas de antemano— hasta completar el recorrido.

1.2. Descripción del problema

Dado un conjunto ordenado de N waypoints $\{(x_1, y_1), \dots, (x_N, y_N)\}$, el robot debe desplazarse desde su posición actual hasta cada waypoint sucesivo, considerándolo alcanzado cuando la distancia euclídea es inferior a una tolerancia ϵ . El único sensor disponible para el control es la **odometría** del robot, que proporciona la pose (x, y, θ) en el plano.

El problema se descompone en dos subproblemas:

1. **Orientación:** dirigir el eje longitudinal del robot hacia el waypoint objetivo.
2. **Avance:** mantener una velocidad lineal que permita alcanzar el objetivo de forma estable.

2. Fundamentos teóricos

2.1. Modelo cinemático del robot

El Pioneer P3DX utiliza tracción **diferencial**: dos ruedas motrices de radio R separadas una distancia $2K$ (distancia entre ejes). El modelo cinemático relaciona las velocidades angulares de las ruedas (ω_i, ω_d) con las velocidades lineal v y angular ω del robot:

$$v = \frac{(\omega_i + \omega_d) R}{2}, \quad \omega = \frac{(\omega_d - \omega_i) R}{2K} \quad (1)$$

La cinemática inversa permite obtener las velocidades de rueda a partir de las velocidades deseadas del robot:

$$\omega_i = \frac{v(1 - \gamma)}{R}, \quad \omega_d = \frac{v(1 + \gamma)}{R} \quad (2)$$

donde γ es la curvatura normalizada ($\gamma = K_p \cdot \Delta\theta$ en nuestro controlador).

Los parámetros físicos del Pioneer 3-DX son:

- Radio de rueda: $R = 0,097\,518\text{ m}$
- Distancia entre ruedas: $2K = 0,331\text{ m}$

2.2. Algoritmo de control: control proporcional de rumbo

La estrategia de control implementada es un **controlador proporcional sobre el error de orientación**. En cada ciclo de control se calcula:

1. El ángulo deseado hacia el waypoint objetivo:

$$\theta_{\text{goal}} = \text{atan2}(y_{\text{goal}} - y, x_{\text{goal}} - x) \quad (3)$$

2. El error de orientación, normalizado al intervalo $[-\pi, \pi]$:

$$\Delta\theta = \text{normalize}(\theta_{\text{goal}} - \theta) \quad (4)$$

3. La curvatura comandada mediante ganancia proporcional K_p :

$$\gamma = K_p \cdot \Delta\theta \quad (5)$$

La velocidad lineal v se mantiene constante durante el desplazamiento. La curvatura γ modula las velocidades de cada rueda según las ecuaciones (2), lo que produce un giro proporcional al error angular: a mayor desviación, mayor corrección.

La condición de llegada a un waypoint es:

$$\sqrt{(x_{\text{goal}} - x)^2 + (y_{\text{goal}} - y)^2} < \epsilon \quad (6)$$

donde ϵ es la tolerancia de proximidad.

3. Diseño e implementación

3.1. Arquitectura del sistema

El sistema se compone de un único nodo ROS 2 que se comunica con el simulador Coppeliasim a través de topics estándar. La Tabla 1 resume la arquitectura.

Tabla 1: Nodos y topics ROS 2 del sistema.

Elemento	Tipo	Función
nav_p2p_node	Nodo (C++)	Lee odometría, calcula el control proporcional y publica <code>cmd_vel</code> .
/Pioneer3DX/odom	Odometry	Pose y velocidades estimadas por odometría.
/Pioneer3DX/cmd_vel	Twist	Comandos de velocidad lineal y angular.

El bucle de control opera con un timer de 25 ms (40 Hz). Los waypoints se cargan desde un archivo YAML como parámetros ROS 2.

3.2. Estrategia de control

El flujo de control en cada iteración es:

1. Leer la pose actual (x, y, θ) desde la odometría, extrayendo el ángulo *yaw* del cuaternión.
2. Calcular la distancia euclídea al waypoint actual. Si es menor que ϵ , avanzar al siguiente waypoint.
3. Calcular el error angular $\Delta\theta$ y la curvatura $\gamma = K_p \cdot \Delta\theta$.
4. Obtener las velocidades angulares de rueda (ω_i, ω_d) mediante la cinemática inversa, saturarlas a $\pm\omega_{\max}$.
5. Convertir de vuelta a (v, ω) mediante la cinemática directa y publicar el mensaje `Twist`.

La velocidad lineal se fija a $v = 1,2 \text{ m s}^{-1}$ y la ganancia proporcional a $K_p = 4.0$ en el código. Las velocidades angulares de rueda se saturan a $\pm 0,79 \text{ rad s}^{-1}$.

3.3. Parametrización

La Tabla 2 recoge los parámetros del sistema, configurables desde `nav_params.yaml`.

Tabla 2: Parámetros del controlador P2P.

Parámetro	Valor	Descripción
<code>control_gain</code> (K_p)	4.0	Ganancia proporcional sobre $\Delta\theta$.
<code>tolerance</code> (ϵ)	1,0 m	Distancia mínima para considerar un way-point alcanzado.
<code>time_step</code>	25 ms	Periodo del bucle de control (40 Hz).
<code>max_linear_speed</code>	1,2 m s ⁻¹	Velocidad lineal de cruce.
<code>max_angular_speed</code>	0,79 rad s ⁻¹	Límite de velocidad angular de rueda.
<code>wheel_base</code> ($2K$)	0,331 m	Distancia entre ruedas.
<code>wheel_radius</code> (R)	0,097 518 m	Radio de las ruedas.
<code>loop_waypoints</code>	false	Repetir la secuencia al completarla.

La secuencia de waypoints configurada describe un recorrido rectangular:

$$(0, 0) \rightarrow (10, 0) \rightarrow (10, 10) \rightarrow (-10, 10) \rightarrow (-10, -10) \rightarrow (10, -10) \rightarrow (0, 0)$$

4. Resultados experimentales

4.1. Escenarios de prueba

El controlador se ha evaluado en un entorno abierto de CoppeliaSim con la secuencia de 7 waypoints rectangulares. Se ha probado con y sin bucle (`loop_waypoints`).

4.2. Análisis cuantitativo

Tabla 3: Resultados de la navegación P2P.

Escenario	Resultado	Observación
Recorrido rectangular (7 WP)	✓	Todos los waypoints alcanzados dentro de la tolerancia.
Recorrido en bucle	✓	Repetición estable sin acumulación de error apreciable.

4.3. Análisis cualitativo

El robot se orienta rápidamente hacia cada waypoint gracias a la ganancia $K_p = 4.0$, que produce correcciones agresivas en el rumbo. En los tramos rectos la trayectoria es estable. En los giros de 90° (cambio de waypoint) el robot describe una curva suave, ya que las velocidades de rueda están saturadas y el giro se distribuye a lo largo de varios ciclos de control.

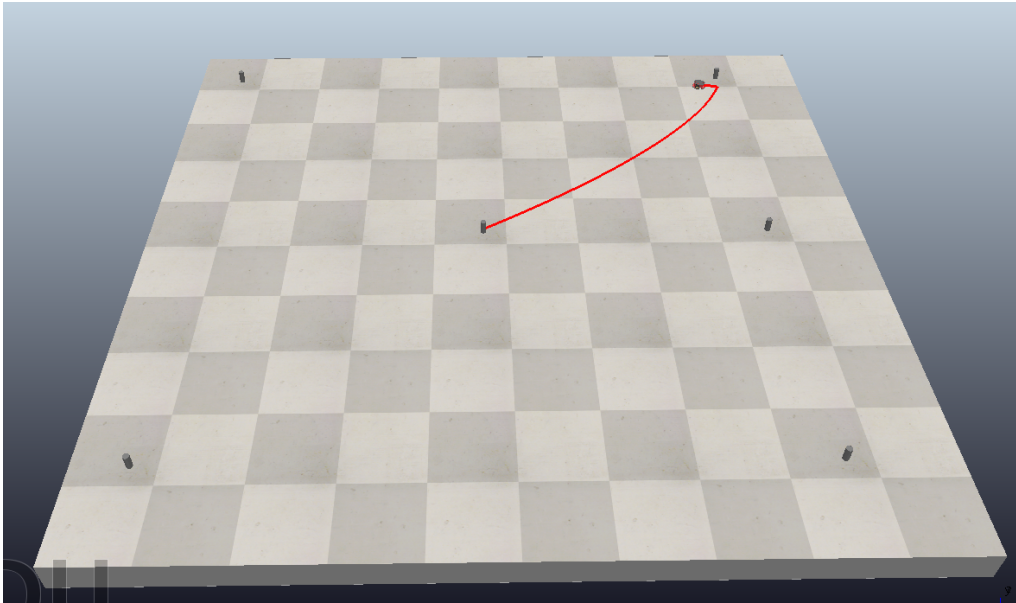


Figura 1: Recorrido rectangular del Pioneer P3DX con control P2P en Coppelia-Sim.

La tolerancia de $\epsilon = 1,0$ m es relativamente amplia, lo que evita oscilaciones al aproximarse al objetivo pero implica que el robot no pasa exactamente por el waypoint.

5. Discusión

5.1. Problemas encontrados y soluciones

Problema 1: oscilaciones en la orientación. Con ganancias bajas ($K_p = 1.0$) el robot corrige lentamente y describe trayectorias con desviaciones laterales significativas. Al aumentar la ganancia a $K_p = 4.0$ la respuesta es rápida, pero si se incrementa más ($K_p > 6$) aparecen oscilaciones porque el error angular cambia de signo en cada ciclo. La solución es el valor $K_p = 4.0$, que ofrece un compromiso entre rapidez y estabilidad.

Problema 2: discontinuidad angular en $\pm\pi$. Sin normalización del error angular, cuando el waypoint está aproximadamente detrás del robot ($|\Delta\theta| \approx \pi$) el controlador puede calcular giros en el sentido incorrecto. La solución es normalizar $\Delta\theta$ al intervalo $[-\pi, \pi]$ en cada ciclo.

Problema 3: saturación de velocidades. Las velocidades angulares de las ruedas calculadas por la cinemática inversa pueden exceder los límites físicos del motor. Se aplica una saturación simétrica $\omega_i, \omega_d \in [-0.79, 0.79] \text{ rad s}^{-1}$, y después se recalculan (v, ω) mediante la cinemática directa para mantener la coherencia.

5.2. Limitaciones

- El controlador no considera **obstáculos**: en entornos no libres el robot colisionará.
- La navegación depende exclusivamente de la **odometría**, que acumula error con el tiempo (*drift*).
- No existe control sobre la **velocidad lineal** en función de la proximidad al objetivo: el robot llega a cada waypoint a velocidad constante.
- Las curvas solo pueden realizarse hacia el lado del waypoint siguiente; no se implementa suavizado de trayectoria entre waypoints consecutivos.

6. Conclusiones

Se ha implementado un sistema de navegación punto a punto para el Pioneer 3DX en C++ sobre ROS 2, basado en un controlador proporcional de rumbo con cinemática diferencial. El sistema:

1. Calcula el error de orientación hacia cada waypoint y lo corrige proporcionalmente mediante la curvatura $\gamma = K_p \cdot \Delta\theta$.
2. Convierte la curvatura en velocidades de rueda, las satura, y recalcula (v, ω) para mantener coherencia cinemática.
3. Gestiona secuencias de waypoints con detección de llegada por tolerancia euclídea.

El resultado es un controlador sencillo y robusto para entornos libres de obstáculos. Como trabajo futuro se podría añadir reducción de velocidad al aproximarse al objetivo, suavizado de trayectorias entre waypoints y fusión con sensores externos para corregir el *drift* odométrico.